YENEPOYA UNIVERSITY

LOW LEVEL DESIGN (LLD) DOCUMENT
ON
Dark Web Monitoring for Agriculture Assets

Student Name: Mohammad Ruwais

Registration No: 23BSCED004

Campus ID: 26099

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Thanziha

Document Type	Low Level Design (LLD)
Version	1.0
Date	April 2026
Domain	Agriculture / Cybersecurity / Dark Web Monitoring

Table of Contents

1. Module: Login & Authentication (app.py – login()) 4
1.1 Purpose 4
1.2 Session State Initialization 4
1.3 Login Function – Step-by-Step 4
1.4 Credential Validation Logic 5
1.5 Configuration Parameters 5
2. Module: Dashboard Shell (app.py – main block) 5
2.1 Purpose 5
2.2 Sidebar Initialization Sequence 5
2.3 Navigation Routing Logic 6
2.4 Logout Sequence 6
3. Module: Threat Scanner (app.py – Monitor Home) 6
3.1 Purpose 6
3.2 Keyword Dictionary 6
3.3 Scan Loop – Step-by-Step 7
3.4 Severity Mapping Table 7
3.5 Threat Classification Logic 8
3.6 Error Handling 8
4. Module: Threat Visualizer (Plotly Bar Chart) 8
4.1 Purpose 8
4.2 Chart Data Construction 9
4.3 Chart Configuration 9
5. Module: Global Threat Analysis 9
5.1 Purpose 9
5.2 Static Data Model 9
5.3 Line Chart Configuration 10
6. Application Data Flow & State 10
6.1 Session State Variables 10
6.2 In-Memory Data Structures 10
7. Deployment & Setup Guide 11
7.1 Prerequisites 11
7.2 Installation 11
7.3 Running the System 11
8. Known Limitations & Future Improvements 11
8.1 Current Limitations 11
8.2 Recommended Enhancements 12

1. Module: Login & Authentication (app.py – login())

1.1 Purpose

The login module is the entry gate of the entire application. It renders a secure credential form, validates submitted credentials, and sets the session_state flag that gates access to all other modules. No dashboard content is rendered until authentication succeeds.

1.2 Session State Initialization

Streamlit's session_state is initialized once at application startup before any page content is rendered:

```
if "logged_in" not in st.session_state:
    st.session_state["logged_in"] = False
```

This guard ensures the key exists on first load and is not re-initialized on subsequent re-runs, preserving the login state across page interactions.

1.3 Login Function – Step-by-Step

The login() function executes the following sequence when called:

- Step 1 – Page Title: Renders '□ Secure Login' as the page heading using st.title()
- Step 2 – Username Input: Renders a text input widget for username collection
- Step 3 – Password Input: Renders a password-type text input (characters masked) for password collection
- Step 4 – Login Button: Renders a 'Login' button; all validation logic executes only when this button is clicked
- Step 5 – Credential Check: Compares submitted username/password against hardcoded credentials
- Step 6a – Success Path: Sets session_state['logged_in'] = True; shows success message; calls st.rerun() to reload the page and route to the dashboard
- Step 6b – Failure Path: Displays st.error('Invalid Username or Password'); session_state remains False

1.4 Credential Validation Logic

```
if st.button('Login'):
    if username == 'admin' and password == 'agri123':
        st.session_state['logged_in'] = True
        st.success('Login Successful!')
        st.rerun()
    else:
```

```
            st.error('Invalid Username or Password')
```

1.5 Configuration Parameters

Parameter	Type	Default Value	Description
username	str	'admin'	Hardcoded valid username; must be externalized in production
password	str	'agri123'	Hardcoded valid password; must use secrets manager in production
logged_in	bool	False	Session flag; controls routing to dashboard vs login screen

2. Module: Dashboard Shell (app.py – main block)

2.1 Purpose

The dashboard shell is the main routing layer that renders after successful login. It initializes the sidebar navigation, handles logout, and conditionally renders one of three modules based on user selection.

2.2 Sidebar Initialization Sequence

```
st.sidebar.title('□ Dashboard Menu')
if st.sidebar.button('Logout'):
    st.session_state['logged_in'] = False
    st.rerun()
app_mode = st.sidebar.selectbox('Navigate To',
    ['Monitor Home', 'Global Threat Analysis', 'Project
Documentation'])
```

2.3 Navigation Routing Logic

The app_mode string returned by the selectbox drives conditional rendering. Only one module block executes per re-run:

app_mode Value	Module Rendered	Primary Function
'Monitor Home'	Threat Scanner + Visualizer	Log input, keyword scan, severity chart
'Global Threat Analysis'	Global Analytics	Monthly attack trend line chart
'Project Documentation'	Static Info Page	Project description text

2.4 Logout Sequence

- Step 1: User clicks 'Logout' button in sidebar
- Step 2: session_state['logged_in'] = False is set
- Step 3: st.rerun() triggers a full Streamlit page reload
- Step 4: Routing condition evaluates False; login() is called

3. Module: Threat Scanner (app.py – Monitor Home)

3.1 Purpose

The threat scanner is the core intelligence engine of the application. It accepts free-text log input from the user, performs case-insensitive keyword matching against a predefined agriculture-sector threat dictionary, assigns severity scores to each detected threat, and triggers visualization.

3.2 Keyword Dictionary

The threat keyword list is defined as a Python list at the module level, making it accessible across all module blocks:

```
keywords = [
    'farm database',
    'agriculture hack',
    'crop data leak',
    'farm iot breach',
    'agriculture credentials'
]
```

NOTE: *Keywords are multi-word phrases. Case-insensitive matching is applied by converting user input to lowercase before comparison. Future versions should support regex patterns and configurable keyword sets loaded from an external config file.*

3.3 Scan Loop – Step-by-Step

- Step 1 – Page Title: Renders '□ Agriculture Dark Web Monitoring System' heading and subtitle
- Step 2 – Log Input Widget: Renders a text area (height=200px) for the user to paste raw log data
- Step 3 – Scan Button: All scan logic executes only when 'Initiate AI Scan' is clicked and input is non-empty
- Step 4 – Lowercase Conversion: user_input.lower() normalizes entire input for case-insensitive matching
- Step 5 – List Comprehension Scan: Iterates over all keywords; retains those present in the normalized input

- Step 6 – Threat Detected Path: If found_threats is non-empty, renders error banner; builds severity DataFrame; renders Plotly bar chart
- Step 7 – Clean Scan Path: If found_threats is empty, renders success banner: 'SCAN CLEAN: No threats detected'

```
if st.button('Initiate AI Scan'):
    if user_input:
        text = user_input.lower()
        found_threats = [word for word in keywords if word in text]
        if found_threats:
            st.error(f'□ {len(found_threats)} Critical Threats
Detected!')
        else:
            st.success('✅ SCAN CLEAN: No threats detected.')
```

3.4 Severity Mapping Table

Each keyword is pre-assigned a fixed severity score. The mapping is positional — the severity list index corresponds to the keyword list index:

Index	Keyword	Severity Score	Classification
0	farm database	9	CRITICAL
1	agriculture hack	8	CRITICAL
2	crop data leak	7	WARNING
3	farm iot breach	10	CRITICAL
4	agriculture credentials	6	WARNING

3.5 Threat Classification Logic

Severity scores are sliced to match only the found keywords using Python list slicing:

```
chart_data = pd.DataFrame({
    'Threat Found': found_threats,
    'Severity Level': [9, 8, 7, 10, 6][:len(found_threats)]
})
```

NOTE: *The positional slicing [:len(found_threats)] assumes found_threats retains the same order as the keywords list. This holds because the list comprehension iterates keywords in order. If keywords are reordered, severity scores must be updated accordingly.*

3.6 Error Handling

Scenario	Detection	Current Handling	Recommended Fix

Empty log input	if user_input check	Scan button does nothing	Show warning: 'Please enter log data'
No threats found	found_threats empty	Show success banner	Log clean scan event for audit trail
Very large input text	No size validation	No limit enforced	Add max character limit to text area
Severity list too short	Slicing [:len]	Silent truncation	Use keyword-to-score dictionary instead

4. Module: Threat Visualizer (Plotly Bar Chart)

4.1 Purpose

The threat visualizer renders an interactive Plotly bar chart that maps each detected threat keyword to its corresponding severity score. It provides security analysts with an immediate visual summary of detected threat severity distribution within the scanned log.

4.2 Chart Data Construction

```
chart_data = pd.DataFrame({
    'Threat Found': found_threats,
    'Severity Level': [9, 8, 7, 10, 6][:len(found_threats)]
})
fig = px.bar(chart_data, x='Threat Found', y='Severity Level',
             title='Threat Analysis')
st.plotly_chart(fig, use_container_width=True)
```

4.3 Chart Configuration

Parameter	Value	Description
x-axis	'Threat Found'	Detected keyword labels displayed along the bottom axis
y-axis	'Severity Level'	Numeric severity score (1–10 scale) on vertical axis
title	'Threat Analysis'	Chart heading rendered above the plot
use_container_width	True	Chart stretches to full dashboard column width
Chart Type	px.bar()	Plotly Express bar chart; interactive hover tooltips included by default

5. Module: Global Threat Analysis

5.1 Purpose

The Global Threat Analysis module provides macro-level visibility into agriculture sector attack trends over time. It renders a Plotly line chart using pre-loaded static monthly data. In future versions this will be replaced by live threat intelligence feeds.

5.2 Static Data Model

```
stats_data = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar'],
    'Attacks': [45, 62, 89]
})
```

Field	Type	Example Values	Description
Month	String	'Jan', 'Feb', 'Mar'	Monthly time axis label for the trend chart
Attacks	Integer	45, 62, 89	Simulated count of agriculture sector dark web attacks per month

5.3 Line Chart Configuration

```
fig2 = px.line(stats_data, x='Month', y='Attacks',
               title='Monthly Attack Trends')
st.plotly_chart(fig2, use_container_width=True)
```

6. Application Data Flow & State

6.1 Session State Variables

Variable	Type	Default	Description
logged_in	bool	False	Authentication gate flag. True = dashboard rendered; False = login() called

All other data (scan results, chart DataFrames) is ephemeral — computed fresh on each Streamlit re-run and not persisted anywhere.

6.2 In-Memory Data Structures

Variable	Type	Scope	Description
keywords	list[str]	Module-level	Static list of 5 agriculture threat keywords; shared across all module blocks
found_threats	list[str]	Scan block	Matched keywords from current scan; rebuilt on each button click
chart_data	pd.DataFrame	Scan block	2-column DataFrame: Threat Found + Severity Level; passed to px.bar()
stats_data	pd.DataFrame	Global block	Static 2-column DataFrame: Month + Attacks; rebuilt on each navigation
app_mode	str	Dashboard shell	Active module name from sidebar selectbox; drives if/elif routing

7. Deployment & Setup Guide

7.1 Prerequisites

- Python 3.8+ installed on the deployment machine
- pip package manager available
- Internet connection for initial package installation
- Web browser for accessing the Streamlit dashboard

7.2 Installation

Install all required Python dependencies using pip:

```
pip install -r requirements.txt
```

Contents of requirements.txt:

```
streamlit
pandas
plotly
```

7.3 Running the System

```
# Start the Streamlit dashboard
streamlit run app.py

# Dashboard opens at: http://localhost:8501
# Login credentials: admin / agri123
```

8. Known Limitations & Future Improvements

8.1 Current Limitations

- Credentials are hardcoded (admin/agri123) — not suitable for production deployment
- Keyword list is static and hardcoded — requires code changes to add new threat patterns
- Severity scores use positional slicing — if keyword order changes, scores become mismatched
- No scan history or audit log — all results are lost on page refresh or logout
- Global threat data is static demo data — not connected to any live threat intelligence feed
- No role-based access control — all authenticated users have identical permissions
- Single-user Streamlit design — does not support concurrent multi-user access

8.2 Recommended Enhancements

Enhancement	Description
Keyword Config File	Load keywords and severity scores from an external YAML/JSON config file to allow updates without code changes
Scan History Storage	Persist scan results to SQLite or Firebase with timestamp, matched keywords, and severity for audit trails
Severity Dictionary	Replace positional list slicing with a keyword-to-score dictionary to prevent score mismatches
Live Threat Feed	Integrate with dark web intelligence APIs (e.g., DarkOwl, Flashpoint) for real-time threat data
User Authentication	Replace hardcoded credentials with database-backed authentication using bcrypt password hashing
Role-Based Access	Implement analyst vs admin roles with different permission levels for scan and configuration modules
ML-Based Detection	Train a text classification model on real dark web threat data to supplement keyword matching
Alerting System	Add email or SMS notification on CRITICAL threat detection using SMTP or Twilio integration